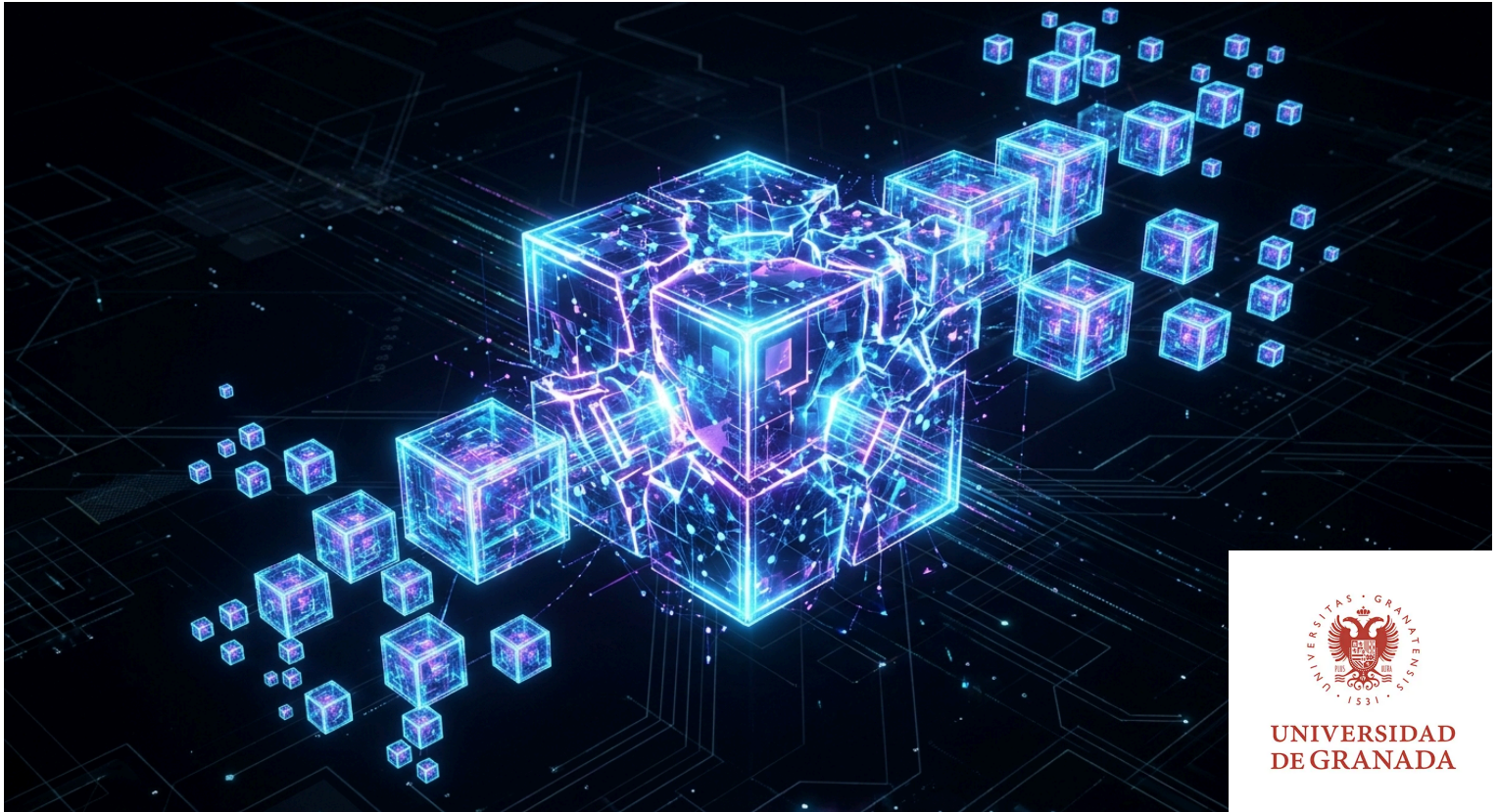


PRÁCTICA 2

ALGORITMOS DIVIDE Y VENCERÁS

Algorítmica, Universidad de Granada



UNIVERSIDAD
DE GRANADA

Realizado por (C1-3):

- Jaime Espín Rodríguez (Problema 1),
 - Marvin Augusto Fonseca Zamora (Problema 2),
 - Omar Segura Arévalo (Problema 1),
 - Francisco González Bailón (Problema 2),
 - Alejandro Moreno Millán (Problema 1).
-

Índice

1. PROBLEMA 1

- 1.1 Enunciado y Análisis
- 1.2 Algoritmo Obvio
- 1.3 Algoritmo Divide y Vencerás
- 1.4 Comparaciones
- 1.5 Conclusiones
- 1.6 Tablas

2. PROBLEMA 2

- 2. 1. Introducción del problema
- 2.2. Diseño del Programa de Pruebas
 - 2.2.1 Tipos de datos evaluados
 - 2.2.2. Genericidad e inicialización de datos
 - 2.2.3. Tipos de pruebas implementadas
 - 2.2.4. Detalles relevantes de la experimentación
- 2.3. Análisis de Resultados
 - 2.3.1. Búsqueda del Umbral Óptimo a Gran Escala (en potencias de 2)
 - 2.3.2. Afinamiento del Umbral
 - 2.3.3. Comportamiento Asintótico con Tamaños Grandes (N en millones)
 - 2.3.4. Comportamiento Asintótico con Tamaños Pequeños (N en decenas de miles)
 - 2.3.5. Rendimiento anómalo de Quicksort frente a Mergesort
- 2.4. Anexos: Tablas
- 2.5 Afinamiento del Umbral
- 2.6 Comportamiento Asintótico con Tamaños Grandes (N en millones)
- 2.7 Comportamiento Asintótico con Tamaños Pequeños (N en decenas de miles)
- 2.8 Rendimiento anómalo de Quicksort frente a Mergesort
- 2.9 Anexos: Tablas

1. PROBLEMA 1

1.1 Enunciado y Análisis

El problema que le ha sido asignado a este grupo es el siguiente:

“Elementos Perdidos: Partiendo de un vector ordenado formado por los k primeros números enteros no negativos $(0,1,2,\dots,k-1)$, se han eliminado n números del mismo (al menos se ha eliminado un número), y los n números que quedan se almacenan en otro vector, con $n < k$. Se pretende encontrar el número más pequeño de los que se han eliminado.

Por ejemplo, si la entrada es $[0, 1, 2, 6, 9, 11, 15]$, el elemento faltante más pequeño es 3.

Si la entrada es $[1, 2, 3, 4, 6, 9, 11, 15]$, entonces el resultado es 0.

Si la entrada es $[0, 1, 2, 3, 4, 5, 6]$, entonces la salida debe ser 7.

Diseñad, analizad la eficiencia (teórica y empírica) e implementad varios algoritmos para resolver este problema: un algoritmo “obvio” y otro algoritmo más eficiente basado en la técnica divide y vencerás (justificad detalladamente su funcionamiento).“

1.2 Algoritmo Obvio

La solución obvia de este problema es bastante trivial. Partimos de una función que recibe el vector modificado, es decir que ya no tiene k primeros números, sino que será $k - n$ elementos (siendo n el número de elementos eliminados, con $n \geq 1$). Partiendo de la premisa de el vector original está ordenado (desde el elemento 0 hasta el elemento $k-1$), este vector modificado también lo estará, por lo que si realizamos un bucle de forma secuencial, preguntando en cada iteración, si el índice del bucle (que está en orden), es igual al elemento del vector. Si esta comparación falla, hay dos posibilidades, que el elemento que falte es el último o que el vector modificado esté totalmente ordenado. Como en el propio enunciado se asume que por lo menos se ha eliminado un elemento, esta posibilidad queda descartada.

Una vez hemos pensado la idea básica, pasamos a realizar la implementación del código. Para tener agrupado el código hemos decidido implementar todas las funciones en una clase *Practica2*. La función *algoritmoObvio* recibe por parámetro una referencia constante a un vector de entero (el vector modificado) y devuelve el entero menor que ha sido eliminado. La implementación que hemos optado es la siguiente:

```
h algoritmoObvio

int Practica2::algoritmoObvio(const vector<int>& vec) const {
    const int size = vec.size(); //Saber el tamaño del vector modificado

    for(int i = 0; i < size; i++){
        if(i != vec[i]) return i; //Elemento encontrado
    }

    //Si llegamos aquí hay dos posibilidades:
    //1. Está sin huecos;
    //2. El hueco es el último elemento.
    //Del enunciado asumimos que sí o sí tiene un hueco, por lo que:
    return size;
}
```

Una vez hemos escrito lo que podría ser la solución, tenemos que poner a prueba antes de mirar la eficiencia teórica y empírica. Para ello, basándonos en el propio enunciado hemos creado la función *testProfe*. Dicha función prueba nuestro algoritmo con los ejemplos dados en el guión de prácticas.

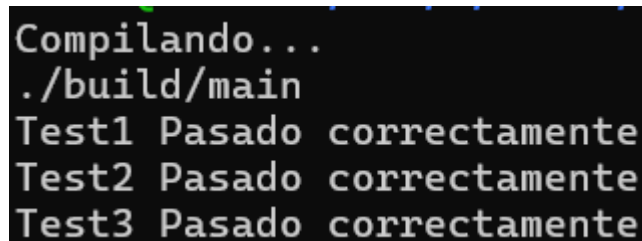


```
void Practica2::testProfe(){
    const vector<int> v1 = {0, 1, 2, 6, 9, 11, 15};
    assert(algoritmoObvio(v1) == 3);
    cout << "Test1 Pasado correctamente\n";

    const vector<int> v2 = {1, 2, 3, 4, 6, 9, 11, 15};
    assert(algoritmoObvio(v2) == 0);
    cout << "Test2 Pasado correctamente\n";

    const vector<int> v3 = {0, 1, 2, 3, 4, 5, 6};
    assert(algoritmoObvio(v3) == 7);
    cout << "Test3 Pasado correctamente\n";
}
```

Procedemos a realizar la ejecución de los tests para saber si es correcto:



```
Compilando...
./build/main
Test1 Pasado correctamente
Test2 Pasado correctamente
Test3 Pasado correctamente
```

Ahora sí, podemos analizar primero la eficiencia teórica del algoritmo básico. Tenemos primeramente una inicialización `const int size = vec.size()` la cual es $O(1)$. Tenemos un bucle for que se ejecuta desde $i = 0$, hasta que lo determine la condición del if interno. Por último, dicha condición `if(i != vec[i])` es también $O(1)$.

El mejor de los casos para este algoritmo ocurre cuando el algoritmo termina lo antes posible, es decir, que el elemento eliminado es el primero `if(0 != vec[0])`. Como el bucle sólo se ejecutaría una vez, y todas las operaciones son $O(1)$, en el mejor de los casos su eficiencia es $O(1)$.

Pero, si en vez de ser el primer elemento fuera el último, estaríamos en el peor de los casos posibles. En este caso, el bucle realizará las n iteraciones completas con una comparación, la eficiencia en el peor de los casos es lineal $O(n)$.

Para realizar las mediciones, hemos tenido que tomar la decisión de elegir qué elemento (o elementos) quitar del vector inicial. Había tres opciones: eliminar el primer elemento (mejor caso), eliminar el último elemento (peor caso) o eliminar un elemento aleatorio (caso promedio). Hemos decidido, para hacer buenas cooperativas posteriormente con el algoritmo de divide y vencerás, elegir siempre que el elemento eliminado es el último.

Evitando crear vectores y eliminando los componentes a mano, hemos decidido implementar un par de funciones que nos sirvan como automatización.

La función *rellenarVector* es una función auxiliar que nos permite, dándole un vector por referencia, un tamaño k y un pivote; rellena de forma secuencial el vector desde 0 hasta k - 1, no introduciendo el valor pivote para que sea ese el eliminado.

```
h rellenarVector

void Practica2::rellenarvector(vector<int>& vec, const int k, const int pivote){
    vec.clear(); //Para limpiar el vector
    vec.reserve(k); //Para evitar copias no necesarias
    for(int i = 0; i < k; i++) { //De 0 a k - 1
        if(i != pivote) vec.push_back(i); //Saltándonos el pivote
    }
}
```

Por último, hemos realizado una función *crearTestAlgoritmoBasico* que recibe únicamente desde main el número k de números positivos que deseamos. Esta función, coge el vector que es una variable miembro privada de la clase y lo rellena con la función anterior siendo el pivote el último elemento. Además, gestiona para hacer las mediciones en *repetidas* veces. Como en la práctica anterior, usamos una clase básica *timer* para obtener el tiempo del reloj preciso del ordenador.

```
h crearTestAlgoritmoBasico

double Practica2::crearTestAlgoritmoBasico(const int k){
    MyTime timer; //Clase auxiliar para el tiempo
    const int repeticiones = 35; //35 repeticiones para el lineal está bien
    double sumador = 0.0; //Para calcular el tiempo promedio

    const int pivote = k; // Siempre el último elemento
    rellenarvector(v, k, pivote); //Rellenamos sin el elemento pivote

    int r; //Para almacenar el resultado
    for(int i = 0; i < repeticiones; i++){
        timer.start();
        r = algoritmoObvio(v);
        timer.end();
        sumador += *timer;
    }

    assert(r == pivote); //Asegurarnos que ha ido bien
    const double time = sumador/double(repeticiones);
    cout << "Test Pasado correctamente (k, pivote) " << k << " " << pivote << " en " << time<< endl;
    return time; //Devolvemos el tiempo a main para guardar los datos
}
```

Con todo esto preparado, en main solo tendremos que crear una variable *p* de tipo *Practica2*, pasamos primeramente de nuevo los test del profesor, y creamos un vector de tamaños para probar el tiempo con distintos tiempos, y los resultados lo guardamos en un archivo .dat para posteriormente usar gnuplot.

```
h main1

int main(){
    Practica2 p;
    p.testProfe();

    vector<int> tams = { //Valores altos, pero con límite para controlar la ram
        100,500,1000,5000,10000,50000,100000, 500000, 1000000, 5000000, 10000000, 25000000,
        50000000, 75000000, 100000000, 150000000, 200000000, 250000000, 300000000
    };

    ofstream basicoFile("data/algorithmoBasico.dat");

    cout << "-----ALGORTIMO BÁSICO-----\n";
    for(const auto t : tams){
        const double time = p.crearTestAlgoritmoBasico(t);
        basicoFile << t << "\t" << time << endl;
    }

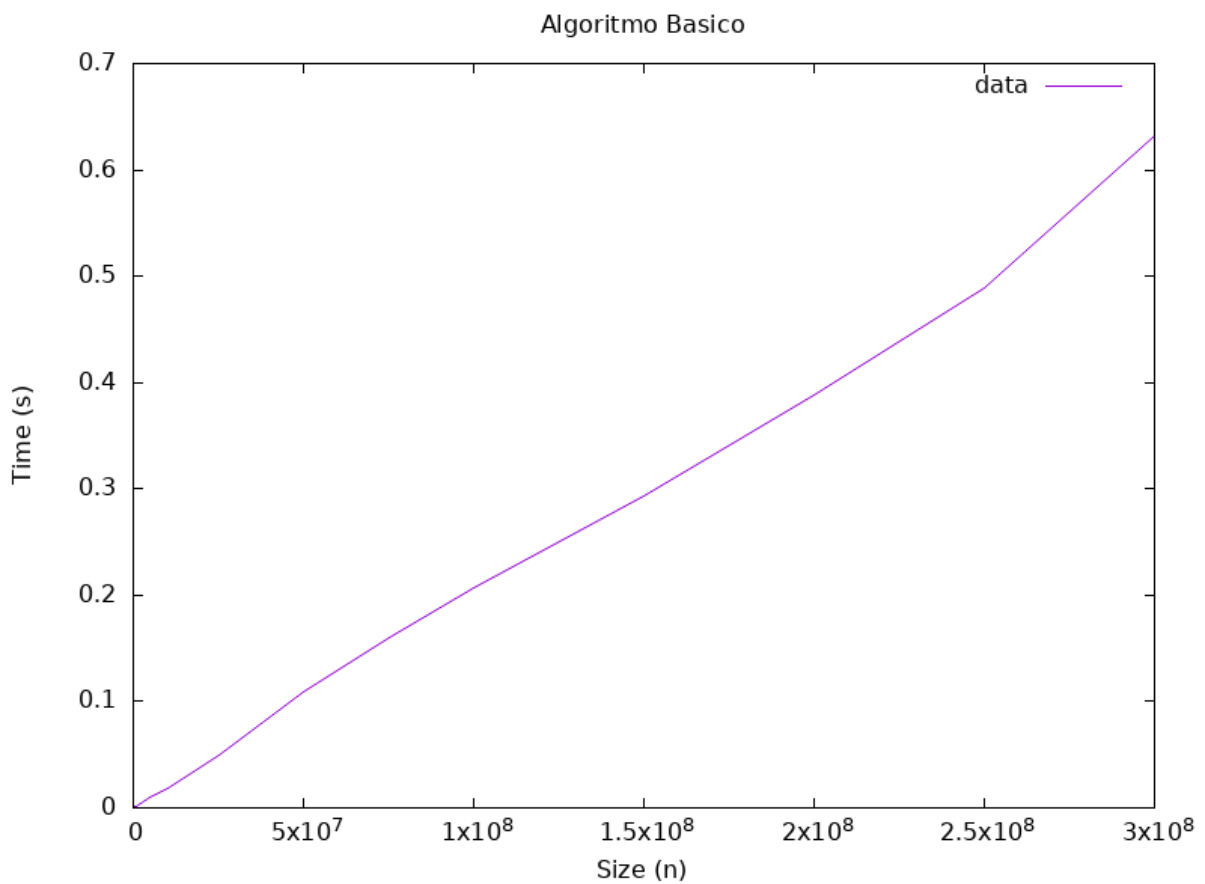
    basicoFile.close();

    return 0;
}
```

Si compilamos y ejecutamos podemos obtener la siguiente respuesta, y con un pequeño script que hemos realizado podemos ver directamente la gráfica asociada:

```
Test1 Pasado correctamente
Test2 Pasado correctamente
Test3 Pasado correctamente
-----ALGORTIMO BÁSICO-----
Test Pasado correctamente (k, pivote) 100 100 en 2.016e-07
Test Pasado correctamente (k, pivote) 500 500 en 1.601e-06
Test Pasado correctamente (k, pivote) 1000 1000 en 1.78997e-06
Test Pasado correctamente (k, pivote) 5000 5000 en 1.46096e-05
Test Pasado correctamente (k, pivote) 10000 10000 en 2.24859e-05
Test Pasado correctamente (k, pivote) 50000 50000 en 9.72035e-05
Test Pasado correctamente (k, pivote) 100000 100000 en 0.000158571
Test Pasado correctamente (k, pivote) 500000 500000 en 0.000895245
Test Pasado correctamente (k, pivote) 1000000 1000000 en 0.00187602
Test Pasado correctamente (k, pivote) 5000000 5000000 en 0.00927769
Test Pasado correctamente (k, pivote) 10000000 10000000 en 0.0185023
Test Pasado correctamente (k, pivote) 25000000 25000000 en 0.0492585
Test Pasado correctamente (k, pivote) 50000000 50000000 en 0.109058
Test Pasado correctamente (k, pivote) 75000000 75000000 en 0.160526
Test Pasado correctamente (k, pivote) 100000000 100000000 en 0.206698
Test Pasado correctamente (k, pivote) 150000000 150000000 en 0.2927
Test Pasado correctamente (k, pivote) 200000000 200000000 en 0.387927
Test Pasado correctamente (k, pivote) 250000000 250000000 en 0.4895
Test Pasado correctamente (k, pivote) 300000000 300000000 en 0.63178
```

Siendo la gráfica del algoritmo obvio:



Vemos como es casi perfectamente una línea recta, reforzando así nuestro análisis teórico del algoritmo. Incluso, lo podemos ver en la propia salida de los datos, si cogemos por ejemplo el tamaño 100.000.000 tarda aproximadamente 0.20 segundos, mientras que para el tamaño 200.000.000 (un por dos) tarda aproximadamente 0.38 segundos (verificando el por dos).

1.3 Algoritmo Divide y Vencerás

Una vez hemos desarrollado el algoritmo básico y hemos visto su rendimiento, creemos que si desarrollamos un algoritmo basado en el paradigma *Divide y Vencerás*, podría ser mucho mejor los resultados. Partiendo de la base de que el vector está ordenado, pensamos que basarnos en una búsqueda binaria sería una posible solución. Como el elemento en cierta posición del vector debe ser igual al índice de dicha posición, si el valor de una posición es mayor que su índice, significa que falta un valor en esa posición o en una anterior. Por tanto, el algoritmo comprobará la posición central del vector; si el valor en ella es mayor al índice, el número perdido se encuentra en la mitad izquierda. En caso contrario, si el valor es igual al índice, todos los números hasta ahí están correctos y el número desaparecido estará en la mitad derecha. De esta forma procedemos dividiendo el problema de forma recursiva hasta que el rango se cierra en una sola posición, encontrando así el número faltante.

Hemos creado la función *DivideyVenceras* que recibe por parámetros: una referencia constante del vector a analizar, la posición del índice izquierdo y la posición del índice derecho. Para generar el algoritmo, primeramente hemos querido identificar cual es el “caso base” para que acabe la recurrencia. En este caso, como hemos abordado anteriormente, nuestro caso base es que el rango (izquierda - derecha) se cierra únicamente en una posición, es decir que ambos índices apunten al mismo elemento (izquierda = derecha), y eso significa que la única posibilidad que existe, es que ese elemento sea el faltante, ya que la mitad izquierda y derecha ya han sido analizadas.

Una vez identificado el caso base, queremos definir las dos subramas posibles. Primeramente, como es una búsqueda binaria, calculamos la mitad exacta del rango (*int mitad = (izquierda + derecha)/2*). Con la variable mitad, accedemos al valor del vector en esa posición, si es mayor que la propia variable mitad, es que está por la izquierda y si no está por la derecha. En ambos casos debemos de llamar a nuestra función de forma recursiva, pero claramente cambiando los índices del rango, si el número perdido está en la mitad izquierda, el nuevo límite derecho será la propia **mitad**; en caso contrario, si está en la derecha, el nuevo límite izquierdo será **mitad + 1**, descartando por completo la parte ya validada.

```
DivideyVenceras

int Practica2::DivideyVenceras(const std::vector<int>& vec, int izq, int drch){

    int mitad = (izq + drch) / 2; //Justo el punto medio

    if (izq == drch) { //Caso Base
        //Este índice es la primera posición donde el valor no coincide con el índice.
        return izq;
    }else if(vec[mitad]>mitad){ //en este caso el número que falta está a la izquierda
        return DivideyVenceras(vec, izq, mitad);
    }else{ //en este caso el nº que falta está a la derecha
        return DivideyVenceras(vec, mitad+1, drch);
    }
}
```

Antes de ponernos a analizar la eficiencia (tanto teórica como empíricamente) tenemos que asegurarnos que nuestra implementación es correcta y resuelve realmente el problema. Para ello hemos adaptado la función *testProfe* para que compruebe también el algoritmo basado en divide y vencerás. En la llamada inicial al algoritmo los índices de izquierda y derecha queremos que sea todo el vector, por lo que será 0 y *v.size()* respectivamente.

A screenshot of a code editor window titled "DivideyVenceras". The code defines a function `void Practica2::testProfe()` which contains three test cases. Each test case defines a vector, calls `algoritmoObvio` and `DivideyVenceras` with assertions, and prints a success message. The test cases use vectors `v1`, `v2`, and `v3` with various elements and sizes.

```
void Practica2::testProfe(){
    const vector<int> v1 = {0, 1, 2, 6, 9, 11, 15};
    assert(algoritmoObvio(v1) == 3);
    assert(DivideyVenceras(v1, 0, v1.size()) == 3);
    cout << "Test1 Pasado correctamente\n";

    const vector<int> v2 = {1, 2, 3, 4, 6, 9, 11, 15};
    assert(algoritmoObvio(v2) == 0);
    assert(DivideyVenceras(v2, 0, v2.size()) == 0);
    cout << "Test2 Pasado correctamente\n";

    const vector<int> v3 = {0, 1, 2, 3, 4, 5, 6};
    assert(algoritmoObvio(v3) == 7);
    assert(DivideyVenceras(v3, 0, v3.size()) == 7);
    cout << "Test3 Pasado correctamente\n";
}
```

Una vez compilado y ejecutado hemos podido ver como estos tests han pasado correctamente. Por lo que ya podemos empezar a analizar la eficiencia teórica.

Este algoritmo divide el vector a la mitad en cada llamada recursiva y sólo explora una de las 2 mitades.

La función divide el problema de tamaño *n* en un subproblema de tamaño *n/2* y realiza un trabajo de combinación y división del vector constante $O(1)$. Esto nos deja con la siguiente recurrencia: $T(n) = T(n/2) + c$

Usando la fórmula maestra , $T(n) = aT(n/b) + cn^k$, podemos resolver la recurrencia identificando los parámetros:

- **a = 1** (número de subproblemas)
- **b = 2** (factor de división)
- **k = 0** (el trabajo de división y combinación es constante)

Como $a = b^k \rightarrow 1 = 2^0$, aplicamos el segundo caso del Teorema Maestro:

$$T(n) = \Theta(n^k \log(n)) = \Theta(n^0 \log(n)) = \Theta(\log(n))$$

Por tanto el tiempo de ejecución es logarítmico, $\Theta(\log(n))$, el algoritmo 'Divide y Vencerás' es más eficiente que el 'Algoritmo Obvio'.

Una vez calculado la eficiencia teórica, vamos a comenzar con el análisis teórico, que será prácticamente igual que para el algoritmo obvio, pero con ciertas diferencias:

```

h crearTestAleatorioAlgoritmoDivideVencerás

double Practica2::crearTestAleatorioAlgoritmoDivideVencerás(const int k){
    MyTime timer;
    int repeticiones;
    if(k < 100000) repeticiones = 5000000; // pequeño
    else if(k >= 100000 && k < 10000000) repeticiones = 2000000; // Medianos
    else repeticiones = 500000; //grande

    const int pivote = k; //El peor caso
    rellenarvector(v, k, pivote);

    int r;
    timer.start();
    for(int i = 0; i < repeticiones; i++){
        r = DivideyVencerás(v, 0, v.size());
    }
    timer.end();

    assert(r == pivote);
    const double time = (*timer)/double(repeticiones);
    cout << "Test Pasado correctamente (k, pivote) " << k << " " << pivote << " en " << time << endl;
    return time;
}

```

A diferencia del test del algoritmo obvio, el número de repeticiones ya no es un valor estático, sino que se ajusta dinámicamente en función del tamaño del vector (k). Dado que este algoritmo es extremadamente rápido ($O(\log n)$), para tamaños de entrada pequeños necesitamos realizar millones de ejecuciones (hasta 5.000.000) para obtener una medida significativa y minimizar el ruido o las fluctuaciones del sistema operativo.

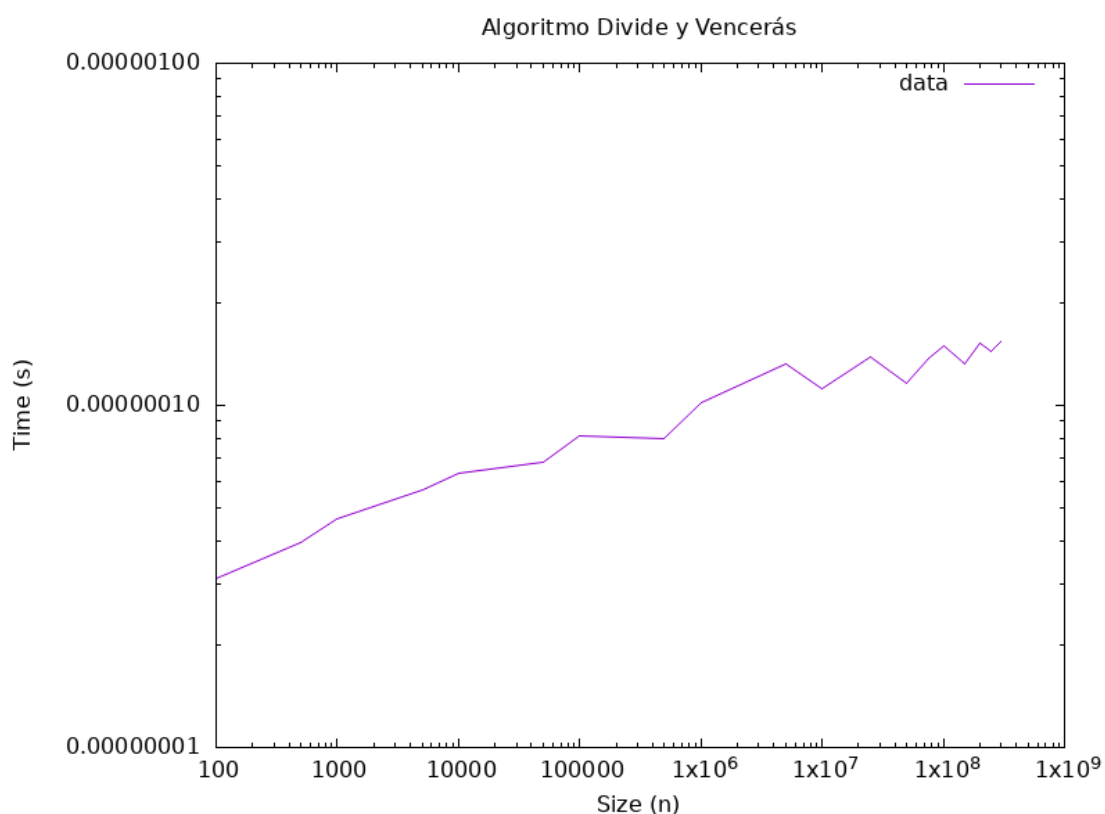
Otra diferencia fundamental radica en la toma de tiempos, que ahora se realiza fuera del bucle de repeticiones. En el algoritmo lineal, el tiempo de ejecución era suficiente para compensar la llamada al reloj en cada iteración; sin embargo, el algoritmo de Divide y Vencerás es tan rápido que el coste computacional de llamar a las funciones del cronómetro (`timer.start()` y `timer.end()`) dentro del bucle sería superior al del propio algoritmo, falseando los resultados. Al medir el bloque completo y dividir por el número de repeticiones, aislamos con precisión el tiempo de cómputo real.

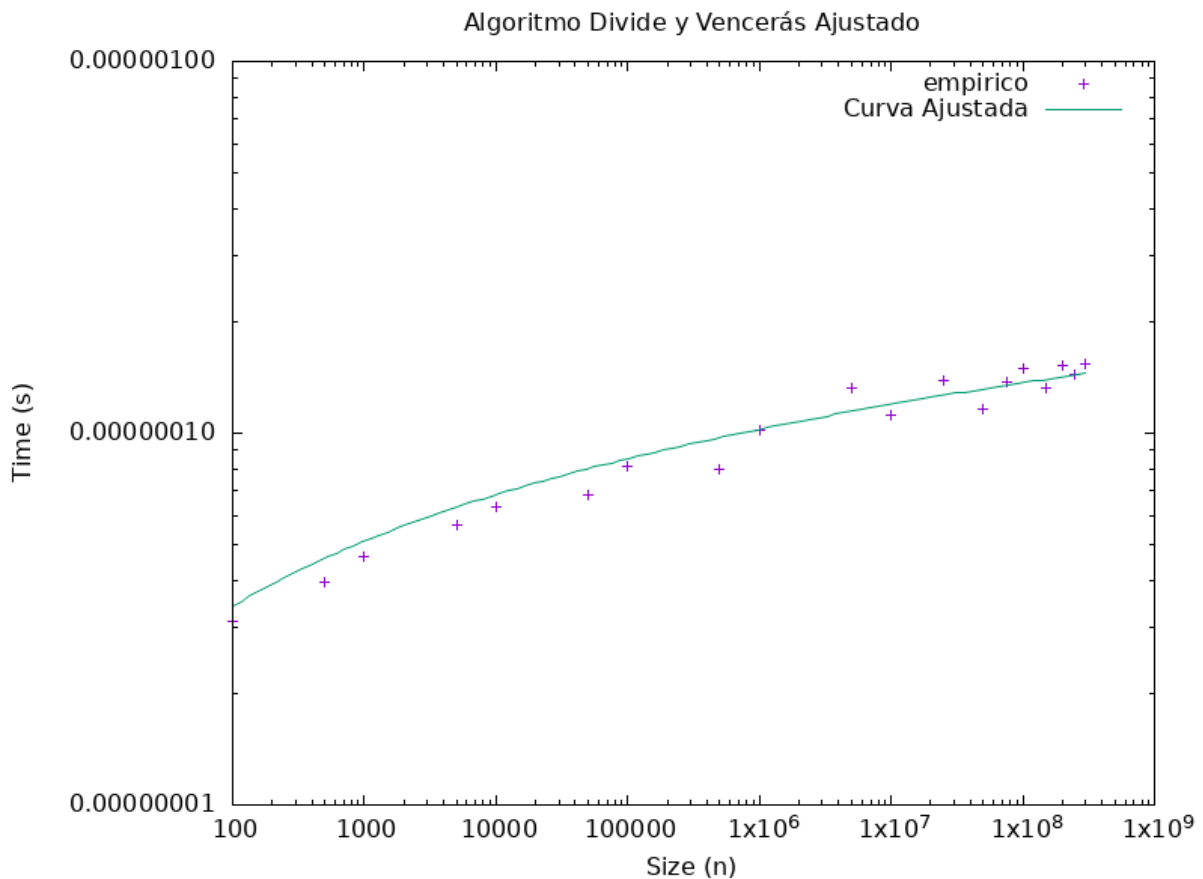
Finalmente, mantenemos la consistencia en el análisis de la complejidad enfocándonos en el **peor caso posible**: situamos el "hueco" o elemento faltante al final del vector (*pivote = k*), obligando al algoritmo a agotar todas las subdivisiones logarítmicas hasta el último nivel de la recursión. El resultado final devuelto sigue siendo el tiempo promedio por ejecución, garantizando una comparativa justa entre ambas soluciones.

Procedemos a ejecutar con los mismos tamaños que habíamos definido para el algoritmo obvio, y obtener los datos en un archivo.dat para realizar las gráficas.

```
Compilando...
./build/main
Test1 Pasado correctamente
Test2 Pasado correctamente
Test3 Pasado correctamente
-----ALGORITMO DIVIDE Y VENCERÁS-----
Test Pasado correctamente (k, pivote) 100 100 en 3.10616e-08
Test Pasado correctamente (k, pivote) 500 500 en 3.98299e-08
Test Pasado correctamente (k, pivote) 1000 1000 en 4.65361e-08
Test Pasado correctamente (k, pivote) 5000 5000 en 5.68164e-08
Test Pasado correctamente (k, pivote) 10000 10000 en 6.31882e-08
Test Pasado correctamente (k, pivote) 50000 50000 en 6.8226e-08
Test Pasado correctamente (k, pivote) 100000 100000 en 8.13242e-08
Test Pasado correctamente (k, pivote) 500000 500000 en 8.02002e-08
Test Pasado correctamente (k, pivote) 1000000 1000000 en 1.02109e-07
Test Pasado correctamente (k, pivote) 5000000 5000000 en 1.32354e-07
Test Pasado correctamente (k, pivote) 10000000 10000000 en 1.11754e-07
Test Pasado correctamente (k, pivote) 25000000 25000000 en 1.38727e-07
Test Pasado correctamente (k, pivote) 50000000 50000000 en 1.16116e-07
Test Pasado correctamente (k, pivote) 75000000 75000000 en 1.36794e-07
Test Pasado correctamente (k, pivote) 100000000 100000000 en 1.49295e-07
Test Pasado correctamente (k, pivote) 150000000 150000000 en 1.32074e-07
Test Pasado correctamente (k, pivote) 200000000 200000000 en 1.5177e-07
Test Pasado correctamente (k, pivote) 250000000 250000000 en 1.43964e-07
Test Pasado correctamente (k, pivote) 300000000 300000000 en 1.53813e-07
```

A partir de estos resultados podemos crear la gráfica en gnuplot:





Para validar los resultados experimentales, hemos contrastado los tiempos obtenidos con el modelo teórico mediante la herramienta Gnuplot (así como hacíamos en la práctica uno). Hemos definido la función de ajuste como $f(x) = a_0 \cdot \log(x)$, buscando el coeficiente que mejor represente nuestra serie de datos. Después de ejecutar el comando fit, el error estándar es de un 2%, por lo que es técnicamente perfecto.

Al representar la gráfica, observamos que prácticamente toda nuestra nube de puntos coincide con la curva de la función $f(x)$. Esto verifica que la implementación del algoritmo de Divide y Vencerás sigue, en efecto, una eficiencia logarítmica, tal como predice el Teorema Maestro.

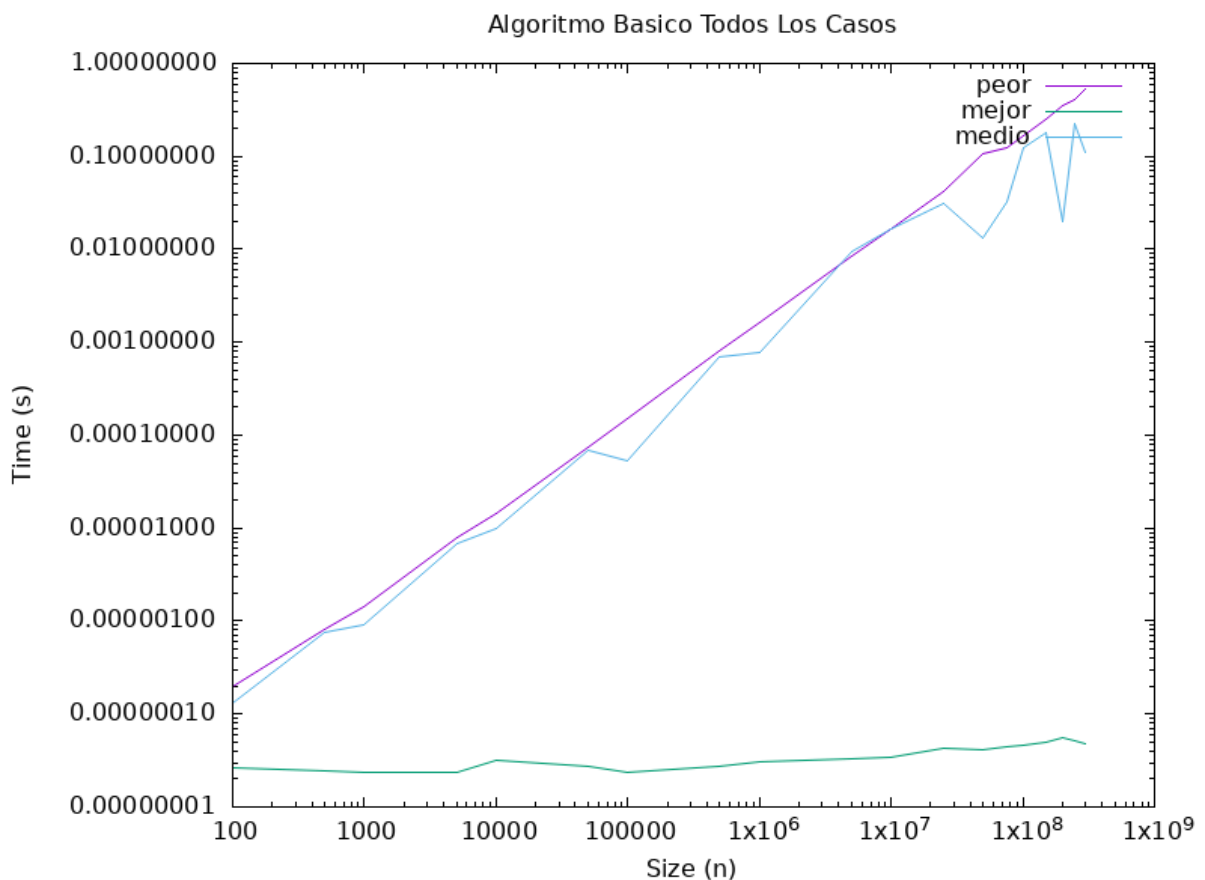
1.4 Comparaciones

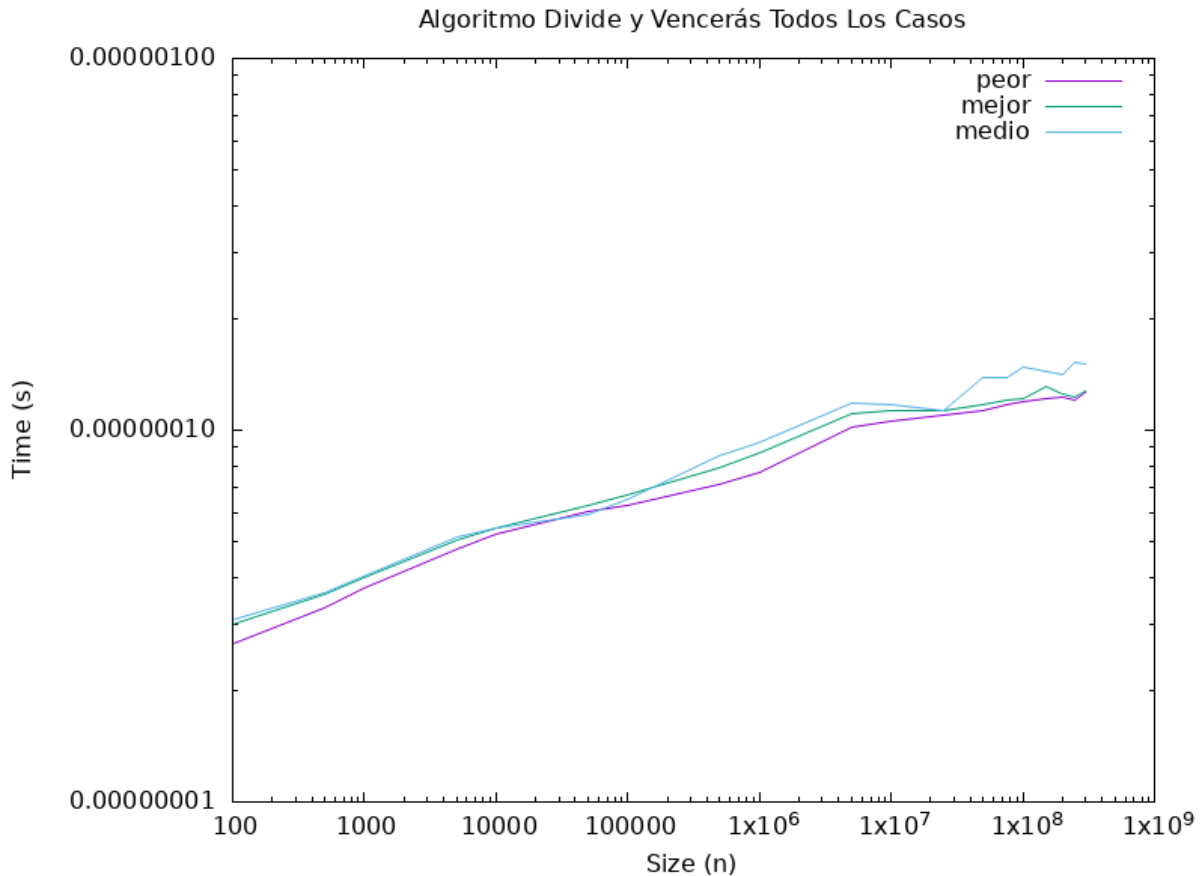
Antes de comparar ambos algoritmos en una misma gráfica para apreciar el comportamiento de ambos. Como hemos utilizado para las pruebas anteriores el peor caso, hemos querido enriquecer el análisis empírico, implementando un método de selección de escenarios mediante un enumerado. Esto permite testear de forma sistemática el mejor caso (hueco al inicio), el peor caso (hueco al final) y un caso promedio mediante la generación de un pivote aleatorio

```
h crearTestAleatorioAlgoritmoDivideVenceras

int Practica2::seleccionarPivote(const int k, const EscenarioPivote escenario) const {
    switch (escenario) {
        case EscenarioPivote::MejorCaso:
            return 0;
        case EscenarioPivote::Medio: {
            static random_device rd;
            static mt19937 gen(rd());
            uniform_int_distribution<int> distribucion(0, k);
            return distribucion(gen);
        }
        case EscenarioPivote::PeorCaso:
        default:
            return k;
    }
}
```

Ahora, en la llamada para los test tenemos que indicarle el escenario que queremos analizar (MejorCaso, Medio, PeorCaso) y se llamará a la función seleccionarPivote para que nos devuelva el pivote correspondiente. Adjuntamos a continuación la tabla de resultados (es una ejecución distinta a los resultados mostrados anteriormente) y gráficas comparativas.





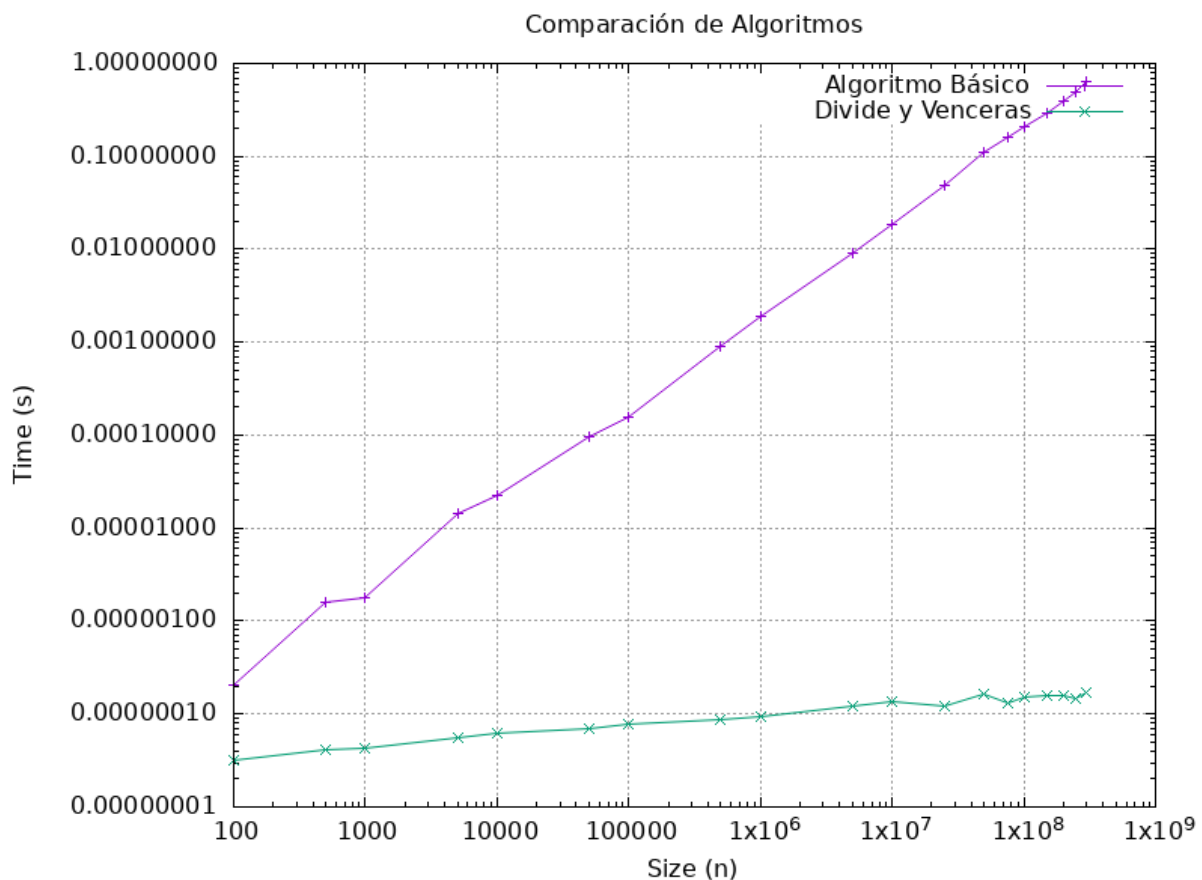
Al analizar las gráficas comparativas de rendimiento, hemos constatado que el comportamiento del algoritmo lineal obedece a las expectativas teóricas: existe una brecha abismal entre el mejor caso (que resuelve el problema en tiempo constante) y el peor caso (con crecimiento lineal), oscilando el caso medio entre ambas cotas.

Sin embargo, los resultados obtenidos para el algoritmo de Divide y Vencerás resultan contraintuitivos a simple vista. Observamos que las líneas correspondientes al mejor, peor y caso medio están prácticamente solapadas, formando una única tendencia logarítmica. Aún más sorprendente es el hecho de que nuestro teórico "mejor caso" (el hueco en la posición 0) llega a registrar tiempos ligeramente superiores al "peor caso" (el hueco al final del vector).

Hemos deducido que este fenómeno se debe a un factor clave de nuestra implementación:

Ausencia de parada temprana (Early Exit): A diferencia del algoritmo lineal, nuestra recursividad no se detiene apenas identifica el hueco. La condición base exige que los límites se solapen (`izq == drch`). Por tanto, independientemente de la posición del número faltante, el algoritmo **siempre** va a dividir el espacio de búsqueda hasta llegar a una sola posición. Esto obliga a realizar exactamente el mismo número de llamadas recursivas en todos los escenarios posibles, unificando sus tiempos de ejecución.

Por último, incluimos una gráfica para poder apreciar las grandes diferencias entre ambos algoritmos, claramente siendo mejor el algoritmo "Divide y Vencerás" con respecto a la eficiencia.



1.5 Conclusiones

Después de realizar este primer problema de la práctica, hemos llegado a la siguiente conclusión:

Por un lado, hemos visto que el "algoritmo obvio" (búsqueda lineal) es muy inestable respecto a la entrada: aunque es imbatible si tenemos la suerte de que el hueco esté en la primera posición, su rendimiento se desploma drásticamente en el peor caso a medida que el tamaño del vector crece.

Por otro lado, la solución basada en **"Divide y Vencerás"** ha demostrado ser inmensamente superior para volúmenes de datos grandes. Aunque en un principio nos sorprendió descubrir que sus tiempos para los casos mejor, medio y peor son prácticamente idénticos (ya que la recursividad siempre divide hasta llegar al caso base), su sólida eficiencia logarítmica, $O(\log n)$, lo convierte indiscutiblemente en la opción ganadora. En definitiva, los resultados de las gráficas confirman que invertir esfuerzo en un buen diseño algorítmico marca una diferencia abismal en el rendimiento real del software.

1.6 Tablas

k	Algoritmo Obvio			Algoritmo Divide Vencerás		
	Peor	Medio	Mejor	Peor	Medio	Mejor
100	1,96E-07	1,31E-07	2,65E-08	2,67E-08	3,09E-08	3,00E-08
500	8,17E-07	7,57E-07	2,43E-08	3,34E-08	3,66E-08	3,63E-08
1000	1,40E-06	9,10E-07	2,33E-08	3,75E-08	4,03E-08	4,01E-08
5000	7,95E-06	6,82E-06	2,33E-08	4,79E-08	5,15E-08	5,07E-08
10000	1,45E-05	9,78E-06	3,21E-08	5,26E-08	5,47E-08	5,46E-08
50000	7,50E-05	7,00E-05	2,71E-08	6,07E-08	5,93E-08	6,25E-08
100000	0,000153	5,37E-05	2,38E-08	6,25E-08	6,49E-08	6,68E-08
500000	0,000807	0,000689	2,69E-08	7,15E-08	8,50E-08	7,91E-08
1000000	0,001651	0,000767	3,11E-08	7,73E-08	9,31E-08	8,72E-08
5000000	0,008457	0,009295	3,28E-08	1,02E-07	1,18E-07	1,11E-07
10000000	0,016683	0,016622	3,41E-08	1,06E-07	1,17E-07	1,13E-07
25000000	0,041554	0,031448	4,33E-08	1,10E-07	1,13E-07	1,13E-07
50000000	0,108359	0,013221	4,17E-08	1,13E-07	1,39E-07	1,18E-07
75000000	0,125063	0,031961	4,42E-08	1,18E-07	1,39E-07	1,21E-07
1E+08	0,167003	0,125353	4,58E-08	1,19E-07	1,48E-07	1,22E-07
1,5E+08	0,249751	0,17724	4,98E-08	1,21E-07	1,43E-07	1,31E-07
2E+08	0,357735	0,01994	5,51E-08	1,23E-07	1,42E-07	1,25E-07
2,5E+08	0,415217	0,22384	5,19E-08	1,21E-07	1,52E-07	1,23E-07
3E+08	0,526461	0,110427	4,77E-08	1,26E-07	1,51E-07	1,28E-07

2. PROBLEMA 2

2. 1. Introducción del problema

Esta parte del informe se centra en el estudio empírico comparativo de dos de los algoritmos de ordenación más representativos: **Mergesort** y **Quicksort**. El problema nos pide analizar el comportamiento de ambos cuando se aplican sobre diferentes tipos de datos (enteros simples y cadenas de caracteres o *strings* con diversas características). Adicionalmente, dado que las implementaciones “*puristas*” de Mergesort y Quicksort resultan ineficientes para problemas de tamaño muy reducido debido a la sobrecarga de las llamadas recursivas, ambos algoritmos se han hibridado utilizando el algoritmo de **Ordenación por Inserción** como caso base.

Este enunciado nos pide determinar el **umbral óptimo** (tamaño del subproblema a partir del cual se deja de dividir y se aplica un algoritmo de ordenación clásico, ordenación por inserción en este caso) para cada algoritmo y tipo de dato (con variaciones de los mismos). Una vez obtenido tal umbral, hemos de comparar los tiempos de ejecución de Mergesort, Quicksort e Inserción en función del tamaño del problema (N), evaluando el impacto de la naturaleza de los datos ordenados (e.g: la longitud de los *strings* o la existencia de prefijos comunes).

2.2. Diseño del Programa de Pruebas

Para poder llevar a cabo el análisis empírico, hemos adaptado y ampliado el código base proporcionado en la práctica anterior, generalizando para cualquier tipo de dato que tenga implementada la comparación ($<$), mecanismos de generación de datos precisos y un sistema de medición de tiempos.

2.2.1 Tipos de datos evaluados

Con el fin de observar cómo afecta la pesadez de la operación de comparación al rendimiento de los algoritmos de ordenación, se han medido y comparado los tiempos de ejecución sobre cinco conjuntos de datos con naturalezas muy distintas:

- **Enteros simples (32 bits):** Representan el caso base o ideal, donde la operación de comparación ($<$) se ejecuta en tiempo constante y mínimo ($O(1)$) a nivel de procesador.
- **Strings de longitud fija pequeña (5 caracteres):** Cadenas de texto cortas. La comparación es algo más costosa que en los enteros, pero sigue siendo muy rápida.
- **Strings de longitud fija grande (20 caracteres):** Cadenas más extensas que obligan a evaluar más caracteres en caso de coincidencia inicial, aumentando el peso de la comparación.

- **Strings de longitud variable (entre 5 y 20 caracteres):** Introducen variabilidad en la memoria y en el proceso de comparación, emulando un escenario de ordenación de palabras de texto real.
- **Strings con prefijo común proporcional a N:** Constituyen el **caso más desfavorable** para la comparación de *strings*. Al compartir un prefijo idéntico muy largo, el operador de comparación debe iterar sobre múltiples caracteres idénticos antes de encontrar la diferencia que rompa esa suerte de “empate lexicográfico,” por llamarlo de alguna manera, degradando drásticamente el rendimiento constante asumido teóricamente.

2.2.2. Genericidad e inicialización de datos

Originalmente, los algoritmos proporcionados operaban exclusivamente sobre vectores de números enteros. Para cumplir con los requisitos de la práctica, se procedió a **generalizar las funciones de ordenación** (quicksort, mergesort e inserción) mediante el uso de *templates* (`template <typename T>`) de C++. Esto nos permite que los algoritmos acepten cualquier tipo de dato genérico, siempre y cuando tenga implementado el operador `<`, el cual, por cierto, está soportado de forma nativa tanto por `int` como por la clase `string`.

Para la creación sistemática de estos datos, se han diseñado funciones generadoras e inicializadoras específicas, que más tarde utilizaremos como funtores para la implementación de las pruebas empíricas realizadas:

- `init_enteros`: Rellena el vector con enteros aleatorios acotados por `INT_MAX`. Se ha maximizado el rango de valores para evitar una alta repetición de números idénticos. Esto porque una gran cantidad de elementos repetidos podría provocar particiones muy desequilibradas en Quicksort, llevándolo de forma artificial a su peor caso teórico ($O(N^2)$).
- `init_fijo_pequeno` e `init_fijo_grande`: Hacen uso de la función auxiliar `StringFijo(longitud)` para generar cadenas de caracteres concatenando letras aleatorias hasta alcanzar los 5 o 20 caracteres, respectivamente.
- `init_variable`: Emplea `StringVariable(5, 20)`, la cual primero decide aleatoriamente una longitud en el rango `[5,20]` y posteriormente genera la cadena.
- `init_prefijo`: Emplea la función `CalcularPrefijo(N)`. Si el tamaño del problema (`N`) es mayor a 100.000, el prefijo base “prefijo” se hace crecer iterativamente agregando la subcadena “_se_hace_mas_largo_y”. Posteriormente, a cada elemento del vector se le añade un sufijo aleatorio de 10 caracteres para garantizar que sean distintos.

2.2.3. Tipos de pruebas implementadas

Para sistematizar el estudio y poder construir las gráficas solicitadas, se han implementado tres bancos de pruebas modulares que aíslan las variables independientes del estudio (el Umbral y el Tamaño del problema `N`):

- **Test1 (Umbral fijo, N variable):** Este test mantiene el parámetro global UMBRAL estático y hace crecer el tamaño del vector desde una cota inferior (a) hasta una superior (b) mediante saltos definidos por la variable paso. Es el test utilizado para analizar la complejidad asintótica temporal de los algoritmos frente a diferentes tamaños de entrada.
- **Test2A (N fijo, Umbral variable lineal):** Fija el tamaño del problema a un valor N constante y varía el umbral iterativamente sumando una cantidad lineal (el paso). Resulta idóneo para hacer un *fine-tuning* (ajuste fino) y localizar el valor exacto del mejor umbral en un intervalo muy acotado.
- **Test2B (N fijo, Umbral variable en potencias de 2):** Mantiene N fijo, pero prueba iterativamente los umbrales multiplicándolos por 2 (ej. 2, 4, 8, 16... hasta 256). Este test es fundamental para la primera fase del estudio, ya que permite observar a gran escala en qué orden de magnitud se encuentra el umbral óptimo de hibridación antes de pasar a la búsqueda final.

2.2.4. Detalles relevantes de la experimentación

Para garantizar un mínimo de calidad estadística de los resultados, el código del experimento incluye las siguientes decisiones de diseño:

1. **Promedio empírico:** Los tiempos de ejecución no se basan en una sola corrida. La función `medir_ordenacion` ordena los datos iterando `NUM_PRUEBAS` veces (establecido a 10) y devuelve el tiempo promedio. Para así reducir las anomalías temporales causadas por procesos en segundo plano del sistema operativo.
2. **Preservación del desorden inicial:** En cada iteración del bucle de pruebas, los datos se copian desde el vector original no ordenado (V) a un vector auxiliar (copia). Esto asegura que el algoritmo siempre se enfrente a un caso promedio aleatorio y no a un vector ya ordenado al llegar a la segunda iteración.
3. **Optimizaciones del compilador:** Hemos compilado el código con el *flag* de optimización `-O3` para medir el rendimiento real y mitigar el tiempo de las pruebas. Por lo que dado a que existía el riesgo de que el compilador eliminara el bloque de código de la ordenación al detectar que el vector copia, mencionado anteriormente, no se utilizaba posteriormente. Para evitarlo, se introdujo la instrucción `volatile T dummy = copia[n / 2]`. El uso del cualificador `volatile` fuerza al compilador a efectuar la lectura en memoria, garantizando que el proceso de ordenación no sea eliminado en tiempo de compilación.

2.3. Análisis de Resultados

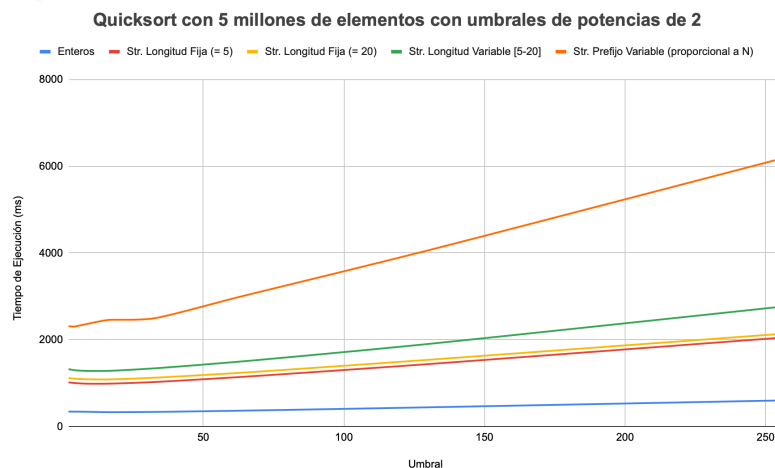
2.3.1. Búsqueda del Umbral Óptimo a Gran Escala (en potencias de 2)

Primero, fijando el tamaño del problema a 5 millones de elementos, se ejecutó un barrido exponencial del umbral, evaluando valores en potencias de 2, desde 2 hasta 256 (**Figuras 1A y 1B** y sus correspondientes tablas).

Como se puede observar en las gráficas, para umbrales bajos (como 2 o 4), el tiempo de ejecución no es óptimo debido a la sobrecarga de las llamadas recursivas (gestión de la pila de ejecución, paso de parámetros, etc.) que no disminuye al tener encima que ordenar vectores de tamaño muy pequeño.

A medida que el umbral supera los valores de 64 o 128, los tiempos de ejecución se disparan (especialmente evidente en *strings* con prefijos, superando los 6s en Quicksort y 5.4s en Mergesort para un umbral de 256). Esto ocurre porque se delegan subproblemas

Figura 1B



demasiado grandes a la Inserción, cuyo coste $O(n^2)$ comienza a dominar sobre el $O(n \log n)$ de los algoritmos Divide y Vencerás.

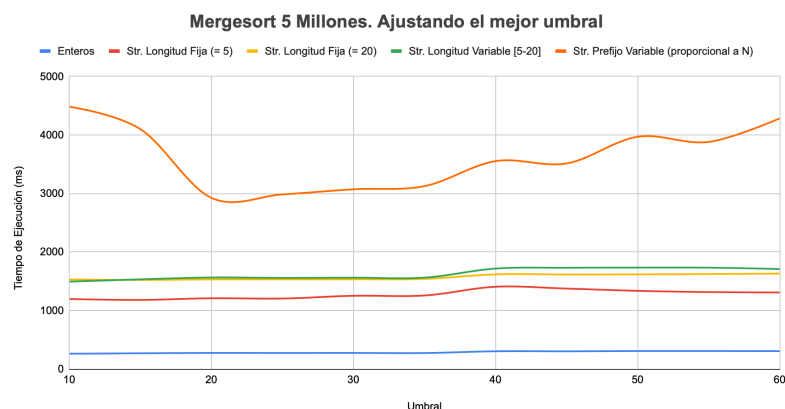
En general, para la mayoría de los tipos de datos, el tiempo mínimo de ejecución se concentra entre los valores 10 y 60 para MergeSort, y 4 y 20 para QuickSort.

2.3.2. Afinamiento del Umbral

Una vez localizado un buen orden de magnitud, se realizaron pruebas de incremento lineal (paso de 5 en 5 para MergeSort y de 2 en 2 para Quicksort) dentro de las zonas de interés anteriormente mencionadas.

Mergesort (Figura 2A y tabla correspondiente): Para enteros simples, el mínimo se alcanza en el umbral 10. Para *strings* de longitud fija, el óptimo oscila alrededor de 15. Para *strings* con prefijo variable, el umbral óptimo sube ligeramente hasta 20. Por lo que concluimos que,

Figura 2A

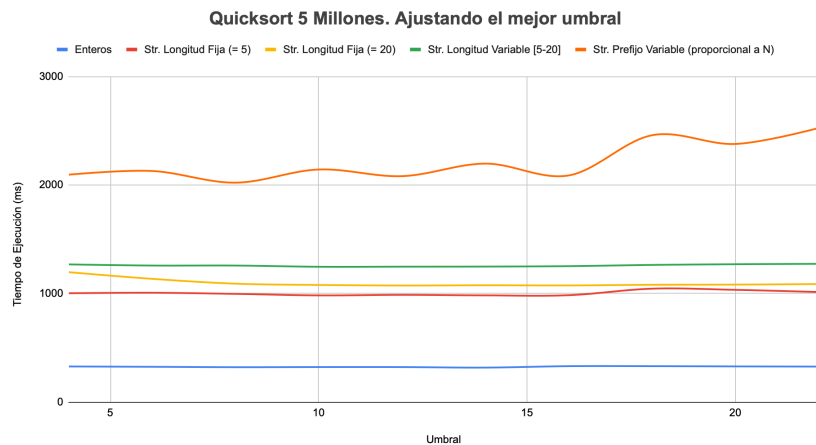


aunque el umbral óptimo fluctúa levemente según el tipo de dato, un umbral general de **14** representa un excelente compromiso para el caso medio en Mergesort.

Quicksort (Figura 2B y tabla correspondiente):

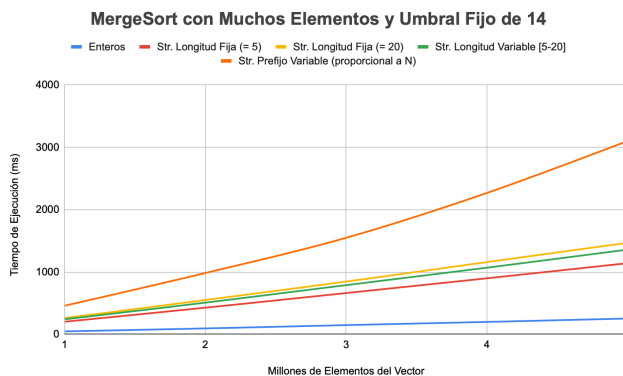
Para enteros, el mejor rendimiento lo encontramos en el umbral 14, mientras que para la mayoría de *strings*, el mínimo absoluto se sitúa en torno al umbral 10 y 12. Para *strings* con prefijo, rinde mejor deteniendo la recursividad un poco antes, rondando el umbral 8. Por ende, en Quicksort, la delegación a Inserción debe hacerse un poco más tarde que en Mergesort, estableciendo un umbral óptimo global de **9**.

Figura 2B



2.3.3. Comportamiento Asintótico con Tamaños Grandes (N en millones)

Una vez fijados los umbrales óptimos obtenidos en la fase anterior, estudiamos el comportamiento de los algoritmos con dichos umbrales pero esta vez fijos con un número de datos variable, probando desde 1 millón hasta 5 millones de elementos (figuras 3A y 3B y sus respectivas tablas).



visualmente las curvas presentan una apariencia casi lineal. Por otra parte, la separación entre las distintas líneas (tipos de datos) nos muestra perfectamente cómo **la constante oculta está mayormente dominada por el coste de la función de comparación**. Los enteros (línea inferior) escalan con una pendiente casi plana. Sin embargo, a medida que el tamaño del *string* aumenta o se vuelve variable, la pendiente se empina.

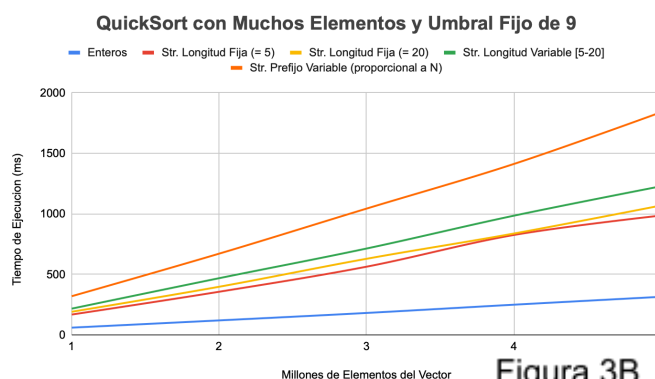


Figura 3B

2.3.4. Comportamiento Asintótico con Tamaños Pequeños (N en decenas de miles)

Las Tablas 4A y 4B muestran el comportamiento de Mergesort y Quicksort para vectores sustancialmente más pequeños, variando desde los 10 mil hasta los 100 mil elementos.

Llama la atención la pronunciada inflexión que hay en el tiempo de ejecución de los *Strings con Prefijo Variable* al acercarse a los 90.000 - 100.000 elementos en ambas gráficas. Este pico se debe a la implementación del

Figura 4A

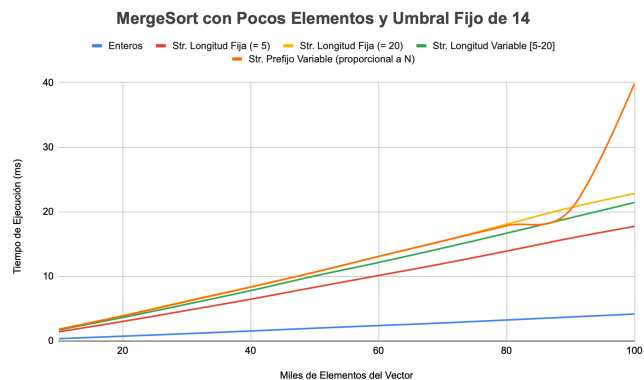
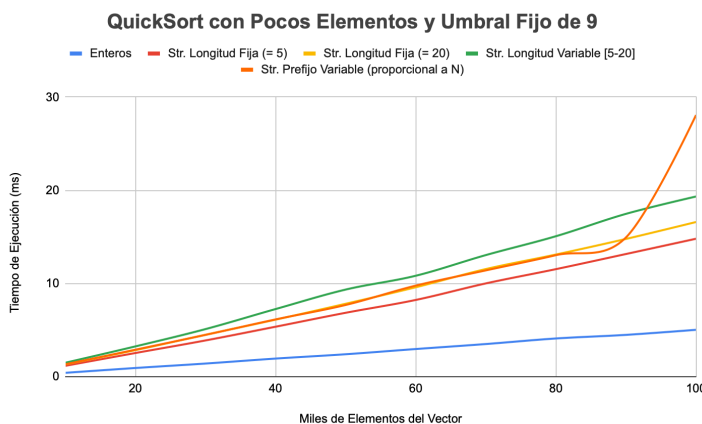


Figura 4B



generador de datos (CalcularPrefijo), el cual está programado para añadir concatenaciones adicionales al prefijo base justo cuando $N \geq 100.000$ (con un límite de 10 añadiduras de prefijos en el caso de evaluar millones de elementos con motivo de evitar que, si se le da un N lo suficientemente grande, el programa escale lo suficiente como para intentar reservar demasiada memoria; error de

principiante que sufrimos durante la realización de esta práctica).

Mientras que Mergesort y Quicksort ordenaban 100 mil elementos en una escala de milisegundos, el algoritmo de Inserción requiere decenas de segundos (el eje Y de la gráfica está en segundos, llegando a rebasar los 40 segundos para el caso de *strings* con prefijo). Confirmando así empíricamente que Inserción es un algoritmo extremadamente ineficiente para conjuntos de datos moderados o grandes.

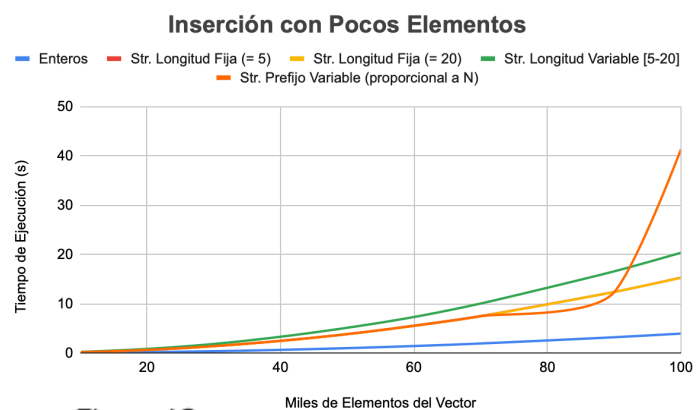


Figura 4C

2.3.5. Rendimiento anómalo de Quicksort frente a Mergesort

Un fenómeno interesante que notamos durante este estudio empírico fue que Mergesort resultó ser notablemente más rápido que Quicksort al ordenar números enteros, por una razón que desconocemos. Analizando el umbral óptimo de ambos para 5 millones de enteros, Mergesort los ordena en 262.38 ms, mientras que Quicksort requiere 316.26 ms (Tabla 1A y 1B). Sin embargo, en el resto de tipos de datos (*strings*), Quicksort recupera su hegemonía (eg. para *Strings Fijos de 20*, QS tarda 1072.57 ms frente a los 1523.37 ms de Mergesort).

2.4. Anexos: Tablas

El ordenador principal mostrado en el desarrollo de la parte 2 de esta memoria de prácticas fue un Macbook Air M1 con 8GB de RAM, 8 núcleos (4 núcleos de rendimiento y 4 de eficiencia), y 7 núcleos de GPU.

Tabla 1A: MergeSort con 5 Millones de Elementos con Umbrales Potencia de 2

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
2	288.070	1233.020	1594.110	1466.010	4639.830
4	268.787	1198.850	1559.670	1434.740	4461.970
8	264.283	1183.060	1535.190	1417.490	4558.200
16	264.098	1183.710	1520.560	1425.990	4341.750
32	274.103	1206.830	1530.390	1502.980	4111.240
64	303.666	1309.370	1613.700	1692.140	3233.440
128	373.103	1565.870	1855.880	2160.830	4007.500
256	520.500	2099.950	2383.530	2945.940	5487.010

Tabla 1B: QuickSort con 5 Millones de Elementos con Umbrales Potencia de 2

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
2	344.069	1017.920	1117.710	1324.140	2312.190
4	343.582	1003.620	1103.790	1299.040	2304.910
8	341.402	988.790	1090.410	1283.060	2355.790
16	331.359	987.881	1085.170	1283.080	2454.810
32	336.813	1024.360	1123.710	1338.840	2490.370
64	366.232	1145.360	1241.780	1502.690	3007.540
128	440.651	1430.140	1528.850	1890.420	4029.470
256	602.050	2050.400	2138.410	2765.060	6182.780

Tabla 2A: MergeSort con 5 Millones de Elementos. Encontrando el mejor umbral.

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
10	262.386	1197.210	1531.070	1494.670	4485.180
15	269.685	1181.720	1523.370	1533.140	4101.400
20	275.798	1210.140	1532.410	1565.060	2926.790
25	275.009	1205.510	1533.260	1557.950	2985.150
30	275.765	1252.660	1533.990	1562.320	3072.640
35	273.893	1257.980	1541.290	1563.050	3127.380
40	304.047	1406.520	1618.570	1717.760	3554.350
45	303.665	1375.660	1616.580	1731.430	3515.120
50	308.365	1337.060	1618.550	1734.270	3970.380
55	308.315	1316.050	1623.800	1733.180	3882.830
60	306.870	1309.390	1632.380	1710.080	4285.390

Tabla 2B: QuickSort con 5 millones de Elementos. Encontrando el Mejor Umbral.

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
4	326.652	1001.260	1195.530	1267.160	2093.680
6	323.605	1005.760	1134.150	1256.590	2127.970
8	319.957	995.502	1089.620	1256.420	2020.370
10	321.259	981.494	1077.750	1245.340	2141.520
12	321.065	986.491	1072.570	1246.200	2080.330
14	316.266	981.753	1074.860	1247.150	2195.720
16	329.364	983.390	1073.340	1250.970	2086.960
18	329.392	1043.340	1079.360	1262.370	2457.170
20	327.224	1032.660	1080.940	1268.660	2376.840
22	325.580	1012.550	1086.040	1272.500	2524.180

Tabla 3A: MergeSort con Muchos Elementos y Umbral Fijo de 14.

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
1000000	47.7486	202.8430	261.5010	241.5220	459.9830
2000000	95.3221	428.2360	551.6340	509.2410	984.5520
3000000	148.4830	662.7440	847.2680	789.7500	1549.8900
4000000	199.5090	899.6890	1159.8200	1069.9500	2269.0000
5000000	254.3280	1141.9100	1468.9600	1359.7400	3101.9600

Tabla 3B: QuickSort con Muchos Elementos y Umbral Fijo de 9.

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
1000000	60.6908	168.5340	191.4240	217.1260	319.5570
2000000	120.4710	356.8800	398.1590	468.8010	671.3140
3000000	182.0620	564.0460	629.3670	714.0580	1043.6400
4000000	250.3480	826.2570	838.1900	985.5700	1412.9300
5000000	315.7310	987.0970	1070.3300	1229.2700	1841.1600

Tabla 4A: MergeSort con Pocos Elementos y Umbral Fijo de 14

N	Enteros simples (32 bits)	Strings Longitud Pequeña (= 5) de Fija	Strings Longitud Grande (= 20) de Fija	String Longitud Variable (entre 2-20) de	String con Prefijo Variable (proporcional a N)
10000	0.322617	1.383100	1.779810	1.677150	1.774850
20000	0.704467	2.990150	3.876810	3.611490	3.851830
30000	1.098850	4.696000	6.150250	5.658550	6.080920
40000	1.516230	6.438260	8.347480	7.774340	8.313660
50000	1.946900	8.290380	10.627900	10.035200	10.629600
60000	2.348580	10.108600	13.063400	12.131100	13.065000
70000	2.758730	11.946300	15.442900	14.350200	15.452900
80000	3.213010	13.887300	18.061500	16.664300	17.827600
90000	3.679030	15.894600	20.621100	19.038900	20.303100
100000	4.137090	17.723500	22.813100	21.421400	39.873800

Tabla 4B: QuickSort con Pocos Elementos y Umbral Fijo de 9

N	Enteros simples (32 bits)	Strings Longitud Pequeña (= 5) de Fija	Strings Longitud Grande (= 20) de Fija	String Longitud Variable (entre 2-20) de	String con Prefijo Variable (proporcional a N)
10000	0.396817	1.150020	1.303390	1.481410	1.315380
20000	0.915454	2.525270	2.835680	3.236720	2.882680
30000	1.393370	3.875530	4.485350	5.092600	4.472200
40000	1.927390	5.347850	6.107490	7.246050	6.131810
50000	2.392480	6.844190	7.814640	9.337660	7.686380
60000	2.952330	8.223900	9.587010	10.821700	9.761550
70000	3.488880	9.998860	11.539100	13.041900	11.396500
80000	4.085780	11.530400	13.100900	15.066900	13.033700
90000	4.473240	13.157100	14.776000	17.469700	14.943200
100000	5.015860	14.796900	16.591800	19.333100	28.101100

Tabla 4C: Inserción con Pocos Elementos.

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 2-20)	String con Prefijo Variable (proporcional a N)
10000	42.3616	150.2770	153.3150	199.8780	152.3870
20000	152.7570	603.6610	611.2710	807.8940	611.3310
30000	345.9140	1363.0800	1369.5100	1817.4200	1378.6000
40000	613.3040	2465.9800	2429.1600	3283.2100	2437.3800
50000	965.3030	3808.0900	3804.0200	5107.9100	3826.5800
60000	1391.9600	5537.3100	5480.2500	7293.6500	5498.3700
70000	1909.2400	7483.0200	7434.2900	10032.8000	7445.8300
90000	3178.8000	12374.3000	12369.5000	16562.2000	12355.9000
100000	3915.9000	15284.7000	15259.3000	20334.0000	41382.1000

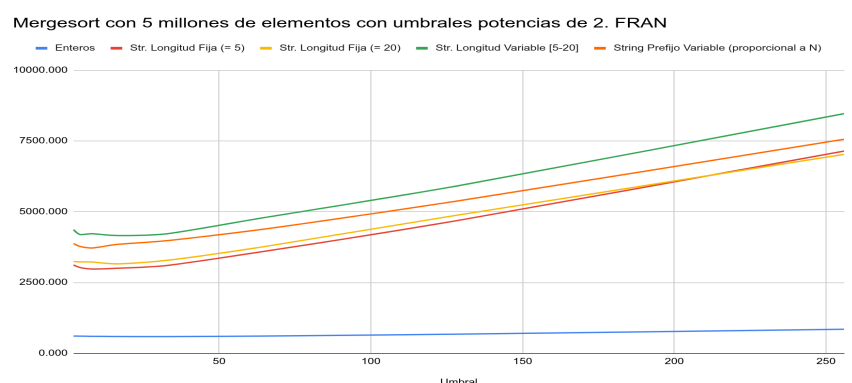
Ahora, observaremos los resultados de dichas pruebas en el ordenador de Fran.

Como se observa en ambas gráficas, el rendimiento óptimo no se encuentra en los umbrales más (2 o 4). Según los datos de la tabla 1C (Mergesort), el tiempo mínimo para la mayoría de las estructuras se halla en el umbral 8. En la tabla 1D (Quicksort), el punto dulce es más marcado, situándose también en torno al umbral 8 (2099 ms para String Fijo de tamaño 5).

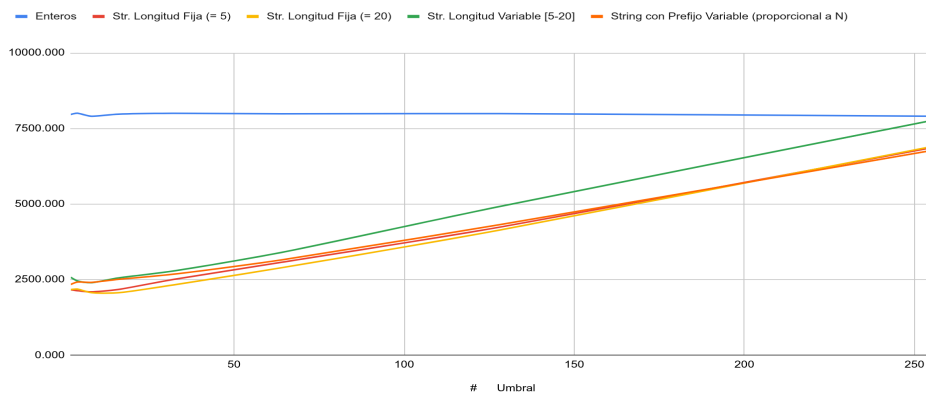
Degradación por Exceso de Delegación (Umbrales > 64)

Al superar el umbral de 64, los tiempos de ejecución sufren un incremento exponencial, especialmente crítico en las cadenas de texto. Al alcanzar el umbral de 256, se observa en la tabla 1D que los String Variables llegan a los 7786 ms, mientras que en la tabla 1C alcanza los 8466 ms.

Este fenómeno se explica porque el coste $O(n^2)$ de la Inserción comienza a dominar sobre el diseño $O(n \log n)$ del algoritmo principal. Delegar problemas de 256 elementos a un algoritmo cuadrático resulta ineficiente, ya que el número de comparaciones y desplazamientos crece mucho más rápido que el ahorro obtenido por eliminar la recursividad.



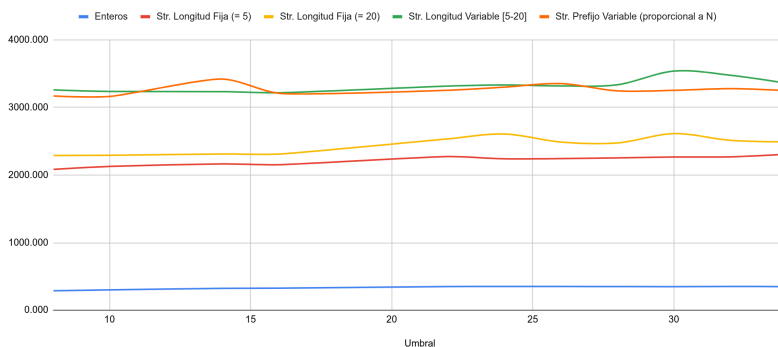
Quicksort con 5 millones de elementos con umbrales potencias de 2. FRAN



2.5 Afinamiento del Umbral

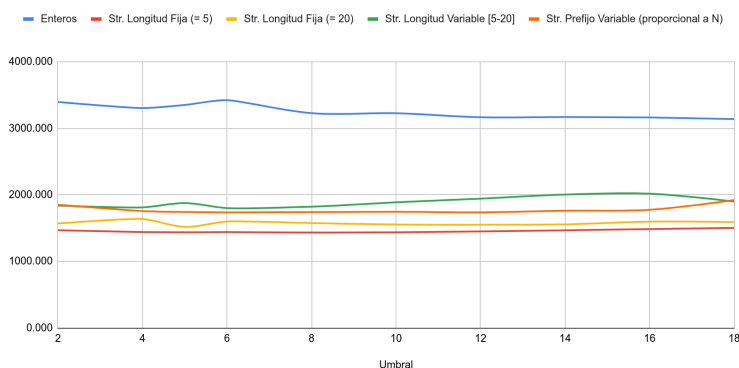
Una vez localizado un buen orden de magnitud en las pruebas anteriores, realizamos pruebas adicionales dentro de las zonas de interés anteriormente mencionadas para encontrar el punto de equilibrio exacto. Como vemos en la Tabla 2C, para enteros simples y strings de longitud fija, el tiempo mínimo absoluto se alcanza de forma contundente en el umbral 8. Para string con prefijo variable, el óptimo se desplaza levemente en el umbral 10. Para string de longitud variable, el rendimiento es mejor delegando a Inserción un poco más tarde, situando su mínimo en el umbral 16. Por lo que podemos concluir que aunque el umbral óptimo fluctúa levemente según la complejidad de la comparación, el umbral 8 representa un buen comportamiento para el algoritmo Mergesort dentro de este algoritmo.

Mergesort 5 millones Buscando el mejor umbral. FRAN



Para el caso de Quicksort (Tabla 2D) el comportamiento difiere según el tipo de dato. Para enteros, el mejor rendimiento lo encontramos en el umbral más alto de la prueba, el umbral 18. Sin embargo, para la mayoría de strings complejos el mínimo absoluto se sitúa de forma muy temprana y marcada en el umbral 6. Para strings de longitud fija, el óptimo oscila entre los umbrales 5 y 8. La delegación a Inserción para strings debe hacerse antes que en Mergesort, estableciendo un umbral óptimo de 6.

Quicksort con 5 millones. Buscar el mejor umbral. FRAN

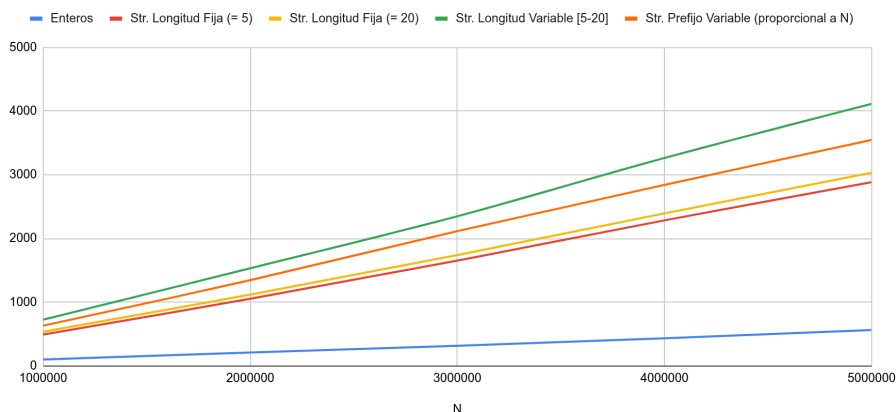


2.6 Comportamiento Asintótico con Tamaños Grandes (N en millones)

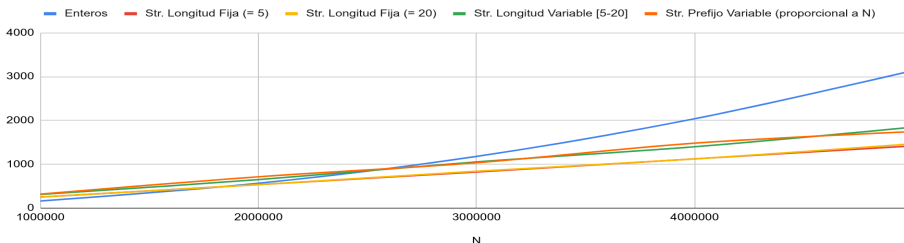
Al analizar la gráfica de **Mergesort** (Tabla 3C), se observa un crecimiento log-lineal. Si tomamos como ejemplo la ordenación de enteros simples, al multiplicar el tamaño de la entrada por 5 (de 1M a 5M), el tiempo de ejecución pasa de 101.6 ms a 564.083 ms. Esto

representa un factor de crecimiento de en torno 5.5x, lo cual encaja a la perfección con la predicción teórica para un algoritmo con complejidad $O(n \log n)$. Todas las curvas de los strings siguen esta misma tendencia.

Mergesort con Umbral Fijo 8. FRAN



Quicksort con Umbral fijo 6 FRAN



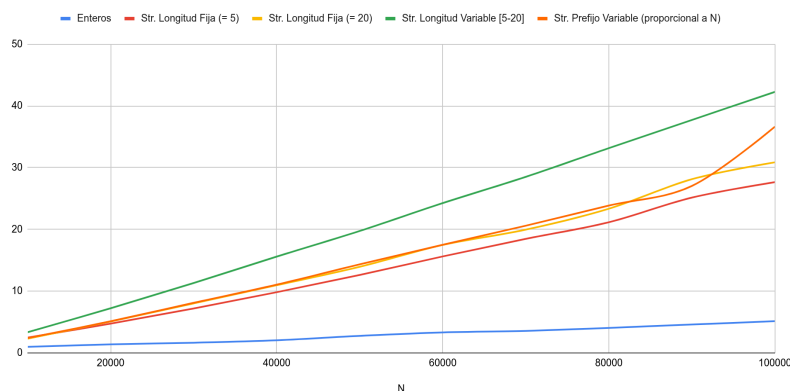
En la gráfica correspondiente a la tabla 3D (Quicksort) podemos ver como por un lado los strings escalan de forma excelente: el string Fijo 5 pasa de 252 ms a 1420 ms (crecimiento esperado de 5.6x). Comparando ambas tablas, Quicksort resulta sistemáticamente más rápido que Mergesort para ordenar strings, llegando a completar los 5 millones de strings de longitud fija en la mitad de tiempo (1420 ms frente a los 2884 ms de Mergesort). Sin embargo, los enteros simples rompe la tendencia de Quicksort. El tiempo pasa de 162.7 ms

a 3137.4 ms al quintuplicar el tamaño N. Esto supone un de casi un 19.3x, lo que evidencia una degradación de la complejidad hacia su peor caso cuadrático $O(n^2)$.

2.7 Comportamiento Asintótico con Tamaños Pequeños (N en decenas de miles)

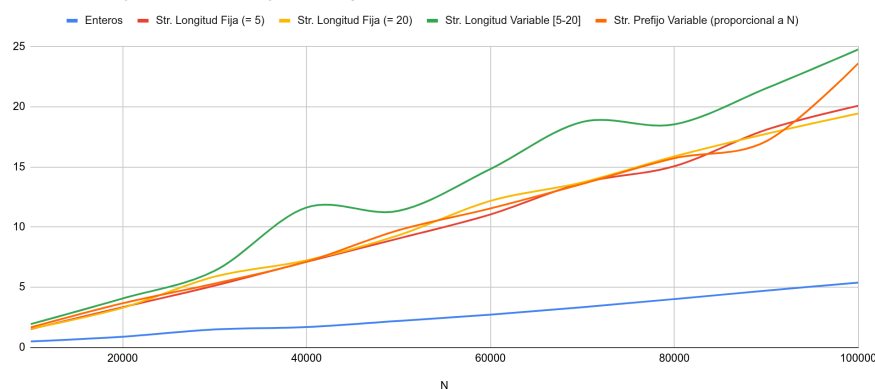
Las tablas 4D y 4E muestran el comportamiento de las versiones con pocos elementos y umbral fijo 8 de Mergesort y Quicksort con umbral fijo 6. Estos datos se contrastan con el rendimiento del algoritmo de Inserción puro, recogido en la tabla 4F.

Mergesort con pocos elementos y umbral fijo de 8. FRAN

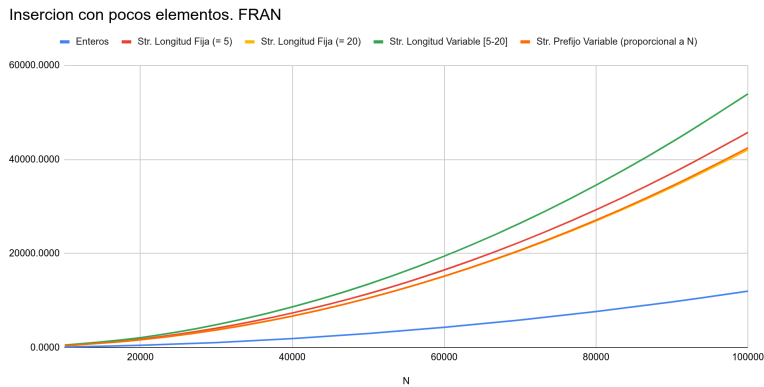


Es notable la pronunciada inflexión que existe en el tiempo de ejecución de los String de Prefijo Variable al alcanzar los 100.000 elementos, pasando por ejemplo de 27 ms a casi 37 ms en Mergesort.

Quicksort con pocos elementos y umbral fijo 6. FRAN



Por otro lado, la comparativa de rendimiento pone de manifiesto la importancia de la complejidad asintótica. Mientras que Mergesort y Quicksort ordenan 100.000 elementos en una escala de escasos milisegundos (tiempos en torno a los 42 ms y 24 ms respectivamente), el algoritmo de Inserción (Tabla 4F) requiere decenas de segundos. Como vemos en los resultados, Inserción llega a rebasar los 42 segundos para el caso de strings con prefijo y se aproxima a los 54 segundos de ejecución total para los strings de longitud variable.



Así, confirmamos empíricamente que Inserción es un algoritmo extremadamente ineficiente y prohibitivo para conjuntos de datos moderados o grandes, lo cual justifica y valida la necesidad de emplear enfoques híbridos que reduzcan su uso exclusivamente a vectores de tamaño muy reducido.

2.8 Rendimiento anómalo de Quicksort frente a Mergesort

Al igual que en los resultados de mi compañero, en los míos sucede igual. Para el caso de los enteros, Mergesort tarda apenas 564 ms, pero Quicksort se “ahoga” y tarda 3137 ms, es decir, casi 6 veces más lento ordenando números.

Para el caso de los strings, Mergesort tarda 2884 ms mientras que Quicksort tarda 1420 ms.

2.9 Anexos: Tablas

Tabla 1C: MergeSort con 5 Millones de Elementos con Umbrales Potencia de 2

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
2	607.262	3119.540	3239.400	4367.330	3874.530
4	604.528	3033.270	3228.500	4197.490	3778.560
8	596.306	2976.390	3224.350	4223.150	3717.030
16	591.915	3001.060	3157.460	4159.600	3841.400
32	588.713	3091.970	3273.570	4212.390	3970.170
64	606.223	3586.630	3752.340	4777.410	4374.920
128	672.854	4678.640	4871.120	5903.300	5371.210
256	849.169	7143.530	7027.230	8466.380	7562.320

Tabla 1D: QuickSort con 5 Millones de Elementos con Umbrales Potencia de 2

#	Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
2		7967.140	2167.030	2174.240	2581.060	2344.930
4		8007.300	2136.890	2190.160	2459.490	2424.580
8		7908.400	2099.670	2073.490	2402.350	2411.860
16		7976.830	2178.700	2069.910	2560.130	2513.250
32		8004.990	2506.550	2324.700	2786.720	2680.890
64		7988.090	3077.770	2898.220	3404.450	3156.870
128		7993.740	4236.510	4140.720	4921.840	4319.130
256		7907.730	6887.590	6925.190	7786.440	6792.950

Tabla 2C: MergeSort con 5 Millones de Elementos. Encontrando el mejor umbral.

Umbral	▼	Enteros simples (32 bits)	▼	Strings de Longitud Fija Pequeña (= 5)	▼	Strings de Longitud Fija Grande (= 20)	▼	String de Longitud Variable (entre 5-20)	▼	String con Prefijo Variable (proporcional a N)	▼
8		288.927		2083.720		2289.190		3260.270		3169.620	
10		301.695		2127.920		2292.430		3236.110		3162.740	
14		324.380		2165.000		2312.450		3234.170		3420.590	
16		327.078		2153.170		2310.690		3217.190		3209.520	
22		351.003		2274.090		2535.810		3316.770		3254.170	
24		351.541		2240.530		2607.580		3333.020		3300.830	
26		351.723		2243.520		2488.150		3318.860		3353.450	
28		350.470		2253.560		2474.950		3335.720		3245.360	
30		349.221		2267.310		2613.530		3538.150		3253.360	
32		352.121		2269.500		2514.160		3476.960		3278.680	
34		349.661		2308.530		2490.350		3357.470		3249.070	

Tabla 2D: QuickSort con 5 Millones de Elementos. Encontrando el mejor umbral.

Umbral	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
2	3397.380	1468.820	1569.720	1839.030	1852.490
4	3305.710	1441.480	1637.160	1811.730	1756.700
5	3352.240	1438.330	1518.370	1877.590	1742.940
6	3423.310	1441.440	1599.860	1801.590	1737.090
8	3227.640	1434.130	1576.820	1823.020	1740.930
10	3226.840	1437.200	1554.080	1888.160	1746.220
12	3166.980	1450.510	1551.190	1943.440	1738.000
14	3170.070	1467.590	1557.280	2004.980	1762.110
16	3163.300	1486.220	1598.210	2018.310	1776.600
18	3138.990	1502.920	1590.850	1899.640	1922.120

Tabla 3C: MergeSort con Muchos Elementos y Umbral Fijo de 8.

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
1000000	101.646	492.921	537.218	726.191	631.176
2000000	211.079	1054.19	1119.57	1532.84	1345.91
3000000	315.735	1652.92	1740.22	2349.13	2116.05
4000000	433.125	2284.25	2393.98	3263.46	2840.95
5000000	564.083	2884.19	3031.72	4112.79	3547.1

Tabla 3D: QuickSort con Muchos Elementos y Umbral Fijo de 6.

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 20)	String de Longitud Variable (entre 5-20)	String con Prefijo Variable (proporcional a N)
1000000	162.732	252.181	251.353	312.333	320.598
2000000	567.647	537.477	536.756	650.616	714.934
3000000	1180.35	822.99	841.299	1055.18	1036.38
4000000	2039.3	1123.99	1121.99	1400.59	1482.85
5000000	3137.49	1420.6	1466.82	1847.4	1745.55

Tabla 4D: MergeSort con Pocos Elementos y Umbral Fijo de 8

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 500)	String de Longitud Variable (entre 5-500)	String con Prefijo Variable (proporcional a N)
10000	0.959871	2.45605	2.29197	3.32953	2.38241
20000	1.36127	4.72861	5.07011	7.21681	5.10934
30000	1.63403	7.18485	7.99993	11.3047	8.097
40000	2.03054	9.81674	10.9487	15.5863	11.0428
50000	2.74084	12.6099	13.9355	19.7381	14.3362
60000	3.30582	15.6133	17.4795	24.2588	17.5047
70000	3.5417	18.4633	19.9534	28.5082	20.592
80000	4.03317	21.1528	23.3446	33.1601	23.8534
90000	4.58252	25.1461	28.1278	37.7287	27.0319
100000	5.12169	27.648	30.8664	42.2769	36.6354

Tabla 4E: QuickSort con Pocos Elementos y Umbral Fijo de 6

N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 500)	String de Longitud Variable (entre 5-500)	String con Prefijo Variable (proporcional a N)
10000	0.489769	1.51281	1.548	1.92774	1.65974
20000	0.881833	3.33759	3.28797	4.06513	3.66325
30000	1.48383	5.12838	5.88094	6.35998	5.30067
40000	1.68458	7.1162	7.23434	11.6325	7.12304
50000	2.19576	9.05342	9.31784	11.3506	9.74402
60000	2.71224	11.0543	12.1832	14.8369	11.5543
70000	3.33836	13.6918	13.7222	18.7559	13.5972
80000	4.01393	15.0666	15.8845	18.5519	15.735
90000	4.71381	18.1064	17.7609	21.5531	17.1477
100000	5.38201	20.0997	19.4571	24.7833	23.6304

Tabla 4F: Inserción con Pocos Elementos.

# N	Enteros simples (32 bits)	Strings de Longitud Fija Pequeña (= 5)	Strings de Longitud Fija Grande (= 500)	String de Longitud Variable (entre 5-500)	String con Prefijo Variable (proporcional a N)
10000	121.3240	441.1620	413.0140	532.3610	415.8950
20000	482.4680	1796.2400	1665.3900	2118.3100	1654.1700
30000	1076.0200	4126.6100	3804.3300	4861.9100	3745.7400
40000	1917.6800	7366.4600	6721.6100	8651.3400	6694.6300
50000	2995.8200	11454.6000	10526.8000	13487.6000	10515.1000
60000	4314.9740	16503.9450	15153.3500	19436.5830	15187.6230
70000	5874.4440	22451.4570	20623.7140	26454.1590	20722.3650
80000	7674.2140	29309.7690	26934.6440	34549.8010	27117.3510
90000	9714.2850	37078.8820	34086.1410	43723.5070	34372.5820
100000	11994.6550	45758.7950	42078.2040	53975.2770	42488.0570